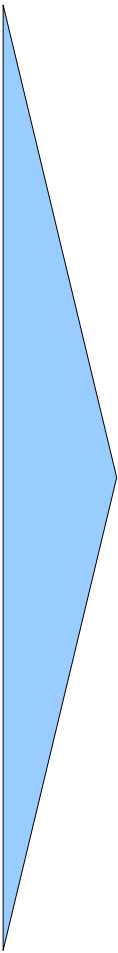
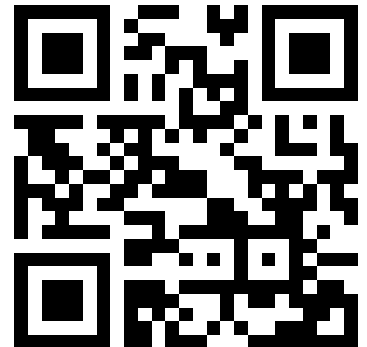


Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>



FreeRTOS Implementation

Problems to solve:

Link the RTOS into the runtime environment

Memory management

Organize system calls out of tasks

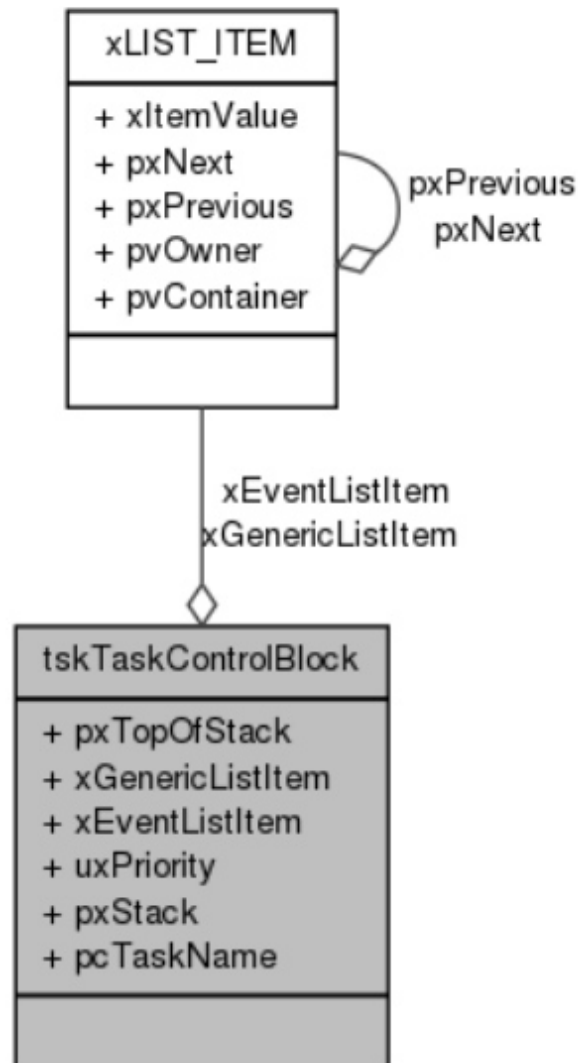
Handle interrupts that make system calls

Debug and trace support

Exceptions and Interrupts

	Exception Number	Exception Type	Priority (Default to 0 if Programmable)	Description
CORE	0	NA	NA	No exception running
	1	Reset	-3 (Highest)	Reset
	2	NMI	-2	NMI (external NMI input)
	3	Hard fault	-1	All fault conditions, if the corresponding fault handler is not enabled
	4	MemManage fault	Programmable	Memory management fault; MPU violation or access to illegal locations
	5	Bus fault	Programmable	Bus error (prefetch abort or data abort)
	6	Usage fault	Programmable	Program error
RTOS	7-10	Reserved	NA	Reserved
	11	SVCall	Programmable	Supervisor call
	12	Debug monitor	Programmable	Debug monitor (break points, watchpoints, or external debug request)
	13	Reserved	NA	Reserved
	14	PendSV	Programmable	Pendable request for system service
	15	SYSTICK	Programmable	System tick timer
I/O	16	IRQ #0	Programmable	External interrupt #0
	17	IRQ #1	Programmable	External interrupt #1
	...	...	...	...
	255	IRQ #239	Programmable	External interrupt #239

Task Control Block (TCB)



SVC Handler Prologue

Load the correct Stack Pointer

```
void vPortSVCHandler( void )
{
    /* Assumes psp was in use. */
    __asm volatile
    (
        #ifndef USE_PROCESS_STACK
            "    tst lr, #4                \n"
            "    ite eq                  \n"
            "    mrseq r0, msp           \n"
            "    mrsne r0, psp           \n"
        #else
            "    mrs r0, psp              \n"
        #endif
        "    b %0                        \n"
        :: "i" (prvSVCHandler) : "r0"
    );
}
```

FreeRTOS's SVC Handler

Organization of a few system services

```
static void prvSVCHandler(    uint32_t *pulParam )
{
    uint8_t ucSVCNumber;
```

```
/* The stack contains: r0, r1, r2, r3, r12, r14, the return address and
xPSR. The first argument (r0) is pulParam[ 0 ]. */
```

```
ucSVCNumber = ( ( uint8_t * ) pulParam[ portOFFSET_T0_PC ] ) [ -2 ];
```

```
switch( ucSVCNumber )
```

```
{
```

```
    case portSVC_START_SCHEDULER:
```

```
        portNVIC_SYSPRI1_REG |= portNVIC_SVC_PRI;
```

```
        prvRestoreContextOfFirstTask();
```

```
    break;
```

```
    case portSVC_YIELD:
```

```
        portNVIC_INT_CTRL_REG = portNVIC_PENDSVSET_BIT;
```

```
/* Barriers are normally not required
but do ensure the code is completely
within the specified behaviour for the
architecture. */
```

```
    __asm volatile( "dsb" ::: "memory" );
```

```
    __asm volatile( "isb" );
```

```
    break;
```

```
    case portSVC_RAISE_PRIVILEGE :
```

```
        __asm volatile
```

```
        (
```

```
            "    mrs r1, control
```

```
            "    bic r1, #1
```

```
            "    msr control, r1
```

```
            ::: "r1", "memory"
```

```
        );
```

```
    break;
```

Start scheduler

Yield

Raise Privilege

Pendsv

Problem:

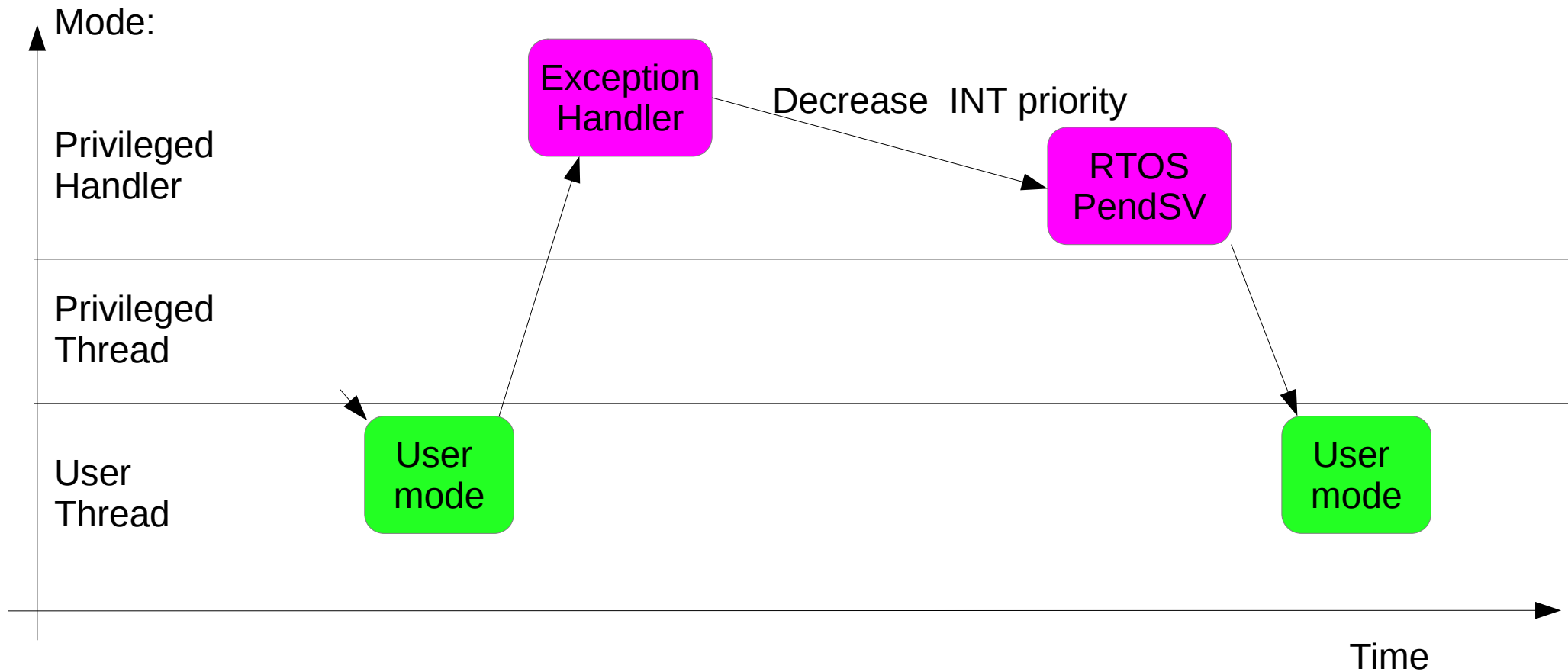
ISRs want to use RTOS services, especially to perform preemptive task switches.

RTOS services need to perform time-consuming operations like checking linked lists. Accordingly RTOS services shall not directly be used within ISRs.

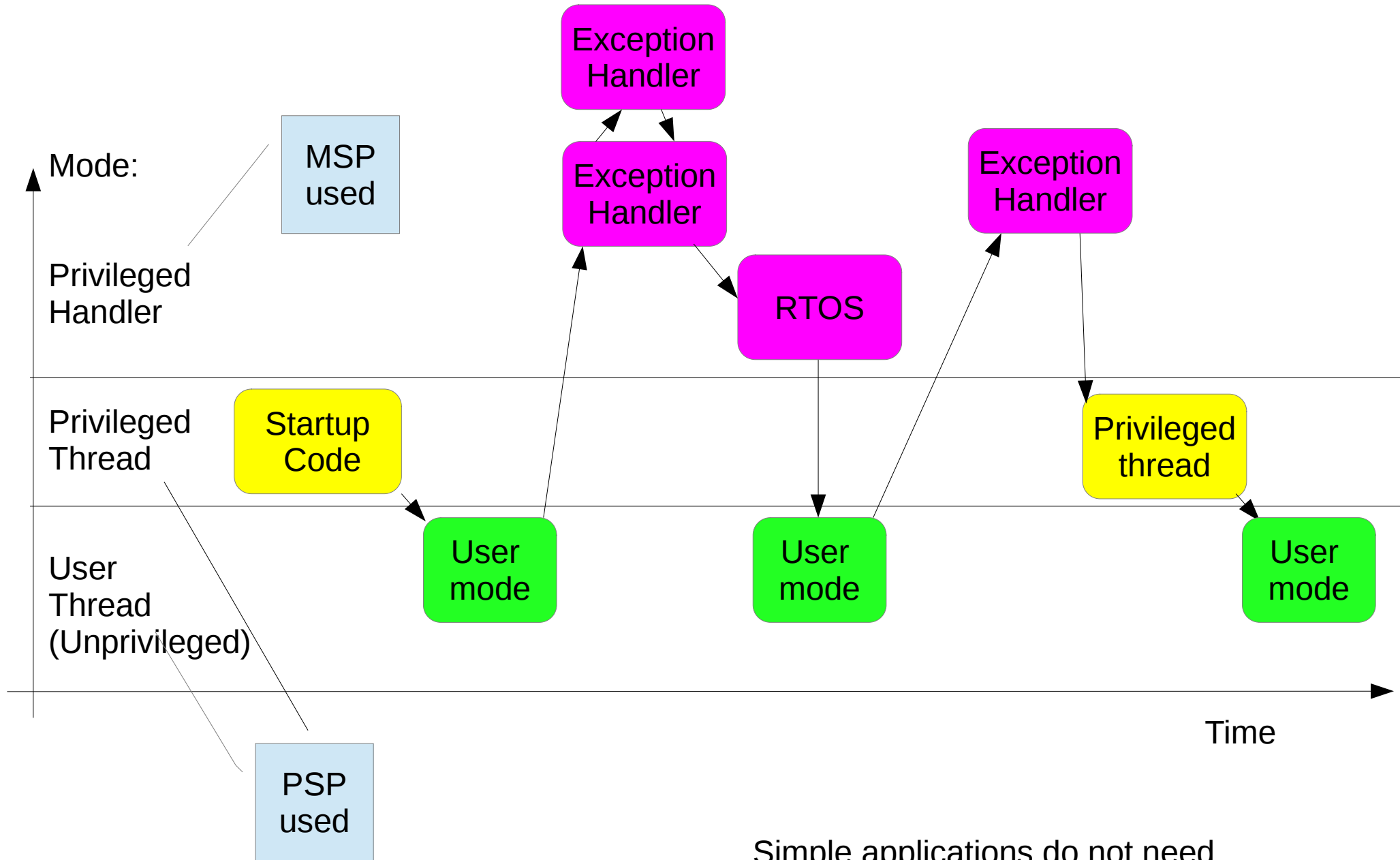
Solution:

ISRs directly do all “fast stuff” that can be done in a few cycles. Necessary RTOS operations will be tail-chained using PendSV. The PendSV Handler is configured to run at lower priority than regular ISRs. Accordingly the PendSV Handler will frequently be interrupted by hardware-triggered ISRs.

Preemptive Task Switch via PendSV



CM3 Operation Mode



Example :

Preemptive Task Switch in FreeRTOS

```
void BUTTON_ISR( void)
{
    portBASE_TYPE TaskWoken = pdFALSE;

    EXTI_ClearITPendingBit(KEY_BUTTON_EXTI_LINE );

    xQueueSendFromISR(EXTIqueue, 0, &TaskWoken);

    portEND_SWITCHING_ISR(TaskWoken);
}

#define portEND_SWITCHING_ISR( xSwitchRequired ) \
    if( xSwitchRequired ) \
        { *(portNVIC_INT_CTRL) = portNVIC_PENDSVSET;}

#define portNVIC_INT_CTRL \
    ( ( volatile unsigned portLONG *) 0xe000ed04 )
#define portNVIC_PENDSVSET          0x10000000
```

Example :

Preemptive Task Switch in FreeRTOS

```
void BUTTON_ISR( void)
{
    portBASE_TYPE TaskWoken = pdFALSE;

    EXTI_ClearITPendingBit(KEY_BUTTON_EXTI_LINE );

    xQueueSendFromISR(EXTIqueue, 0, &TaskWoken);

    portEND_SWITCHING_ISR(TaskWoken);
}

#define portEND_SWITCHING_ISR( xSwitchRequired ) \
    if( xSwitchRequired ) \
        { *(portNVIC_INT_CTRL) = portNVIC_PENDSVSET;}

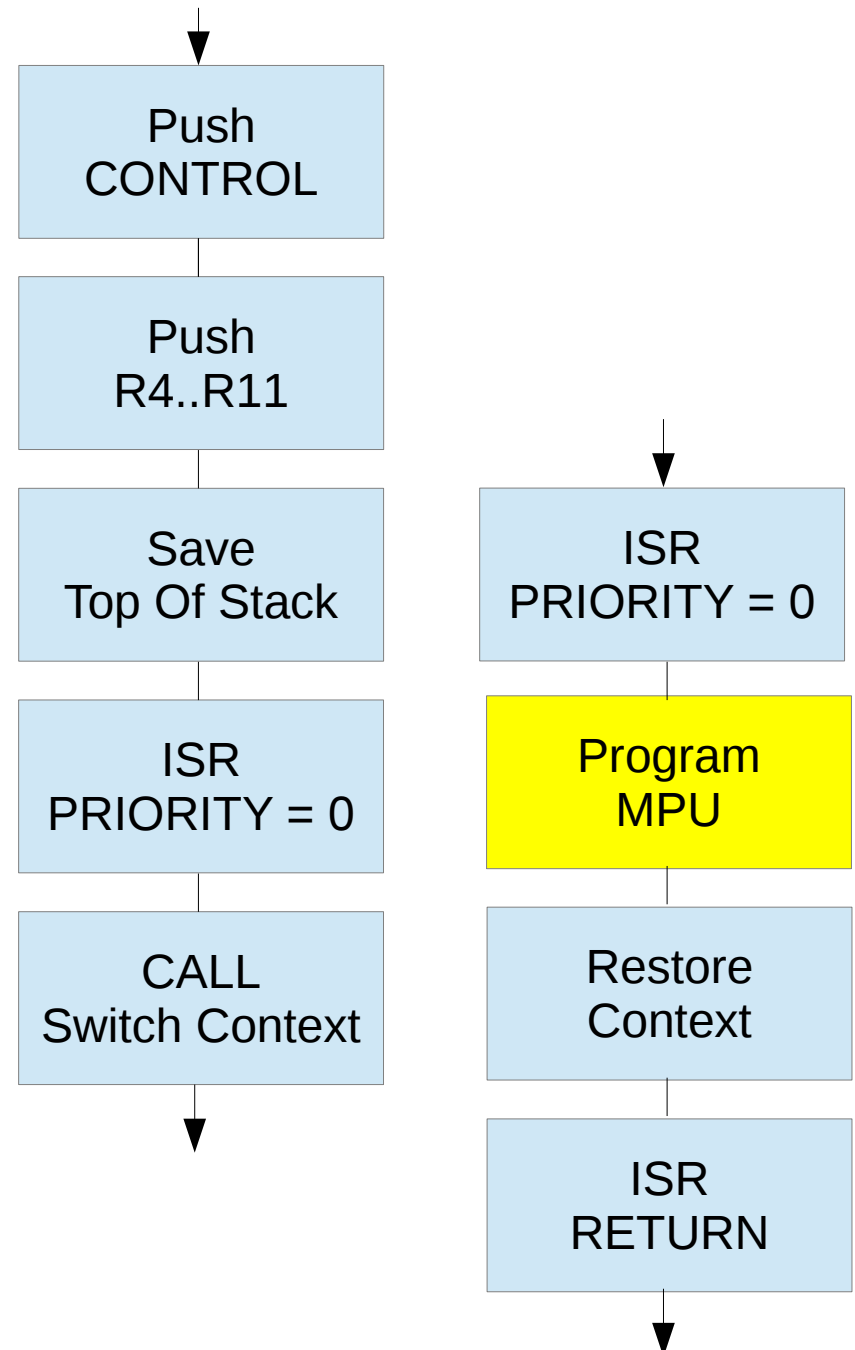
#define portNVIC_INT_CTRL \
    ( ( volatile unsigned portLONG *) 0xe000ed04 )
#define portNVIC_PENDSVSET          0x10000000
```

```

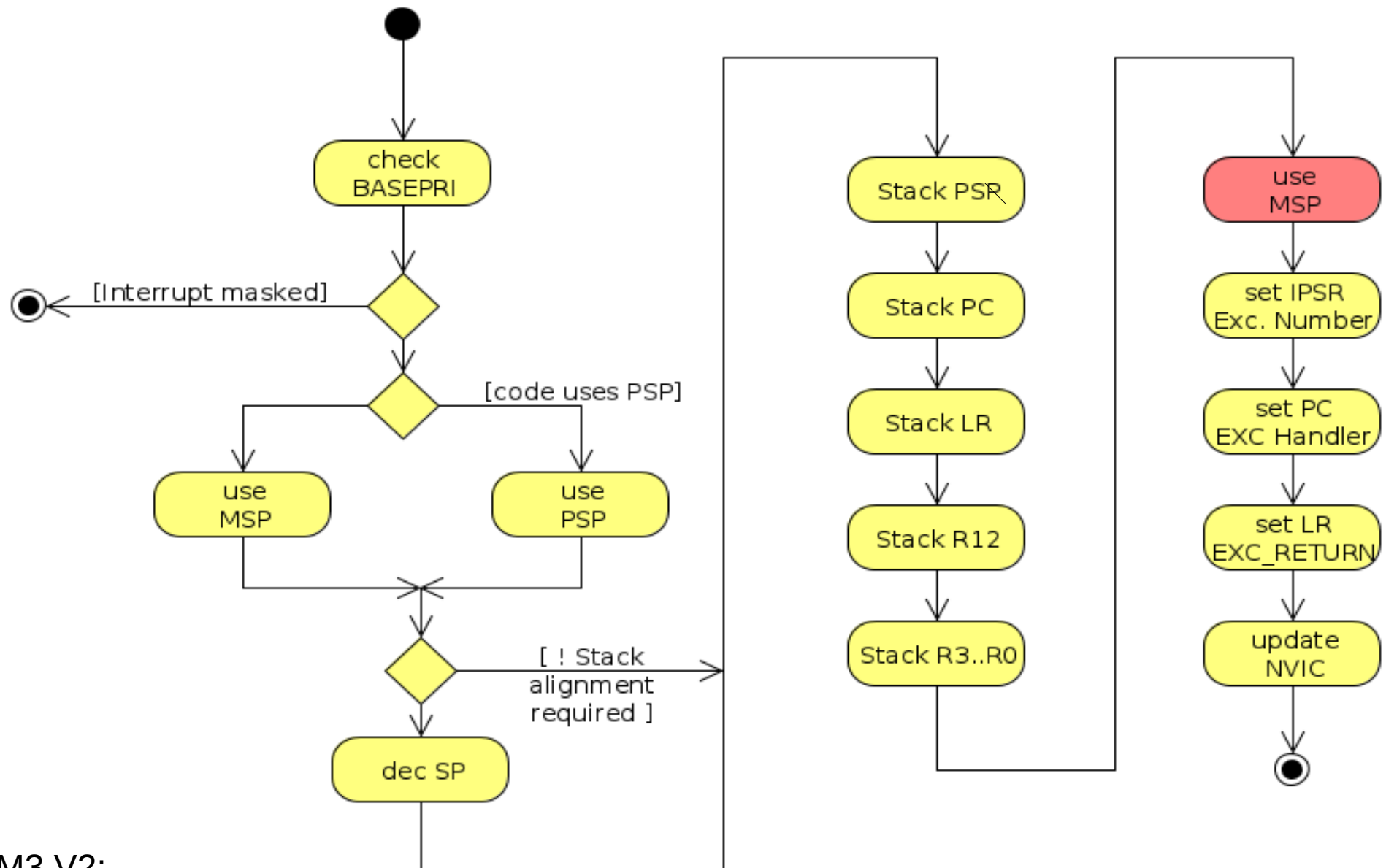
void xPortPendSVHandler( void )
{
    /* This is a naked function. */

    __asm volatile
    (
        "    mrs r0, psp                \n"
        "                                     \n"
        "    ldr r3, pxCurrentTCBConst  \n"
        "    ldr r2, [r3]               \n"
        "                                     \n"
        "    mrs r1, control             \n"
        "    stmdb r0!, {r1, r4-r11}     \n"
        "    str r0, [r2]               \n"
        "                                     \n"
        "    stmdb sp!, {r3, r14}         \n"
        "    mov r0, %0                  \n"
        "    msr basepri, r0              \n"
        "    bl vTaskSwitchContext        \n"
        "    mov r0, #0                  \n"
        "    msr basepri, r0              \n"
        "    ldmia sp!, {r3, r14}         \n"
        "                                     \n"
        "    ldr r1, [r3]                 \n"
        "    ldr r0, [r1]                 \n"
        "    add r1, r1, #4                \n"
        "    ldr r2, =0xe000ed9c           \n"
        "    ldmia r1!, {r4-r11}           \n"
        "    stmia r2!, {r4-r11}           \n"
        "    ldmia r0!, {r3, r4-r11}       \n"
        "    msr control, r3               \n"
        "                                     \n"
        "    msr psp, r0                   \n"
        "    bx r14                       \n"
        "                                     \n"
        "    .align 2                      \n"
        "pxCurrentTCBConst: .word pxCurrentTCB \n"
        "::"i"(configMAX_SYSCALL_INTERRUPT_PRIORITY)
    );
}

```

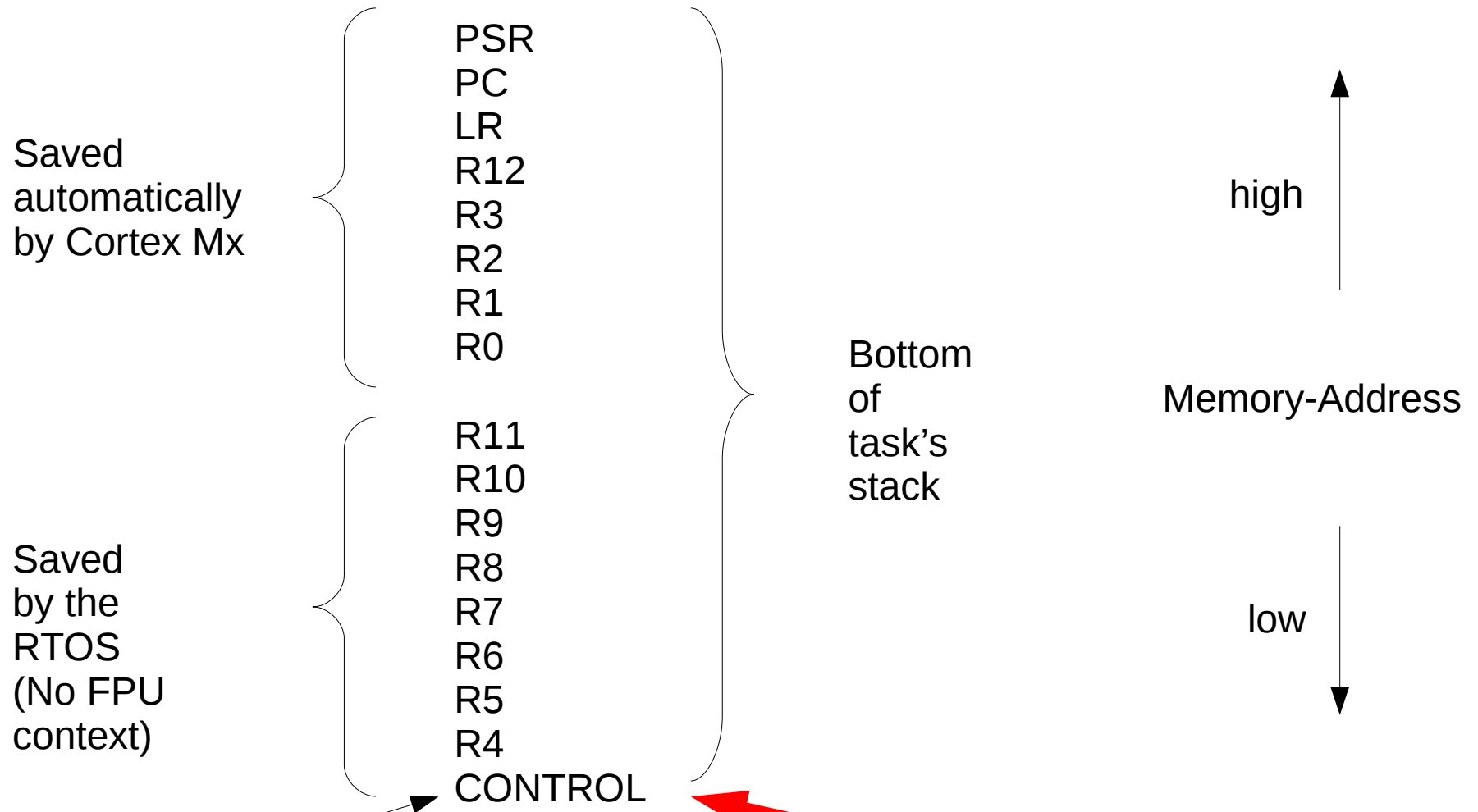


Interrupt / Exception Sequence



CM3 V2:
Stack frame aligned @ 64bits

“Frozen” Task Context



Control maintains the info "privileged" or not

SP points here
SP itself stored within TCB

PendSV

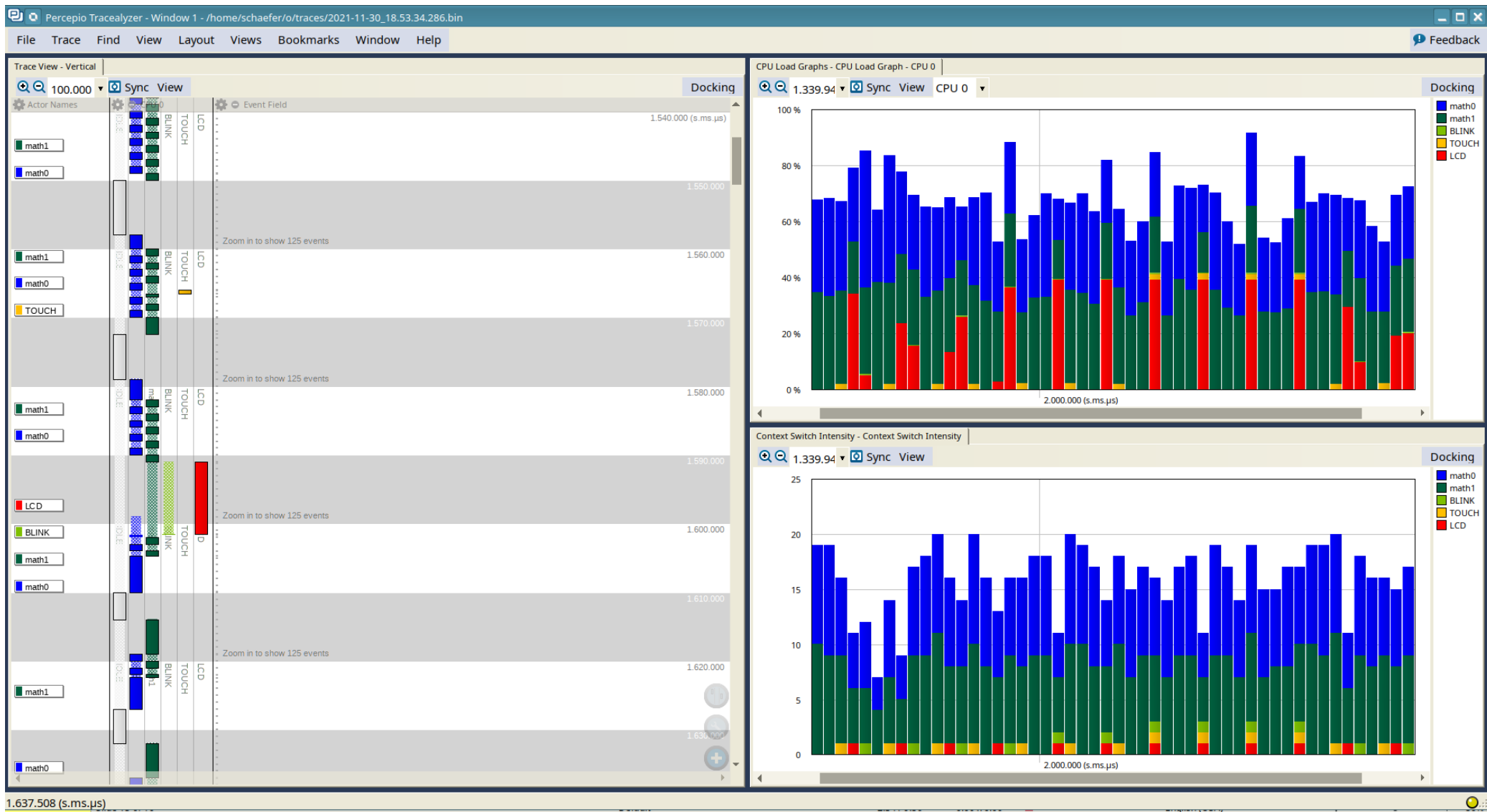
```
void xPortPendSVHandler( void )
{
    /* This is a naked function. */
```

```
__asm volatile
```

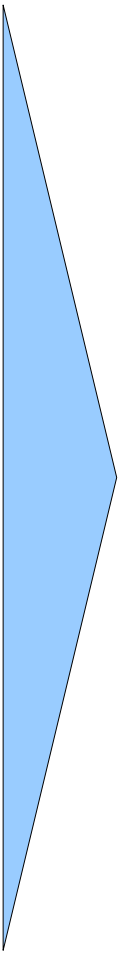
```
(
    "    mrs r0, psp\n"
    "    isb\n"
    "    ldr r3, pxCurrentTCBConst\n"
    "    ldr r2, [r3]\n"
    "    tst r14, #0x10\n"
    "    it eq\n"
    "    vstmdbeq r0!, {s16-s31}\n"
    "    mrs r1, control\n"
    "    stmbd r0!, {r1, r4-r11, r14}\n"
    "    str r0, [r2]\n"
    "    stmbd sp!, {r0, r3}\n"
    "    mov r0, %0\n"
    "    cpsid i\n"
    "    msr basepri, r0\n"
    "    dsb\n"
    "    isb\n"
    "    cpsie i\n"
    "    bl vTaskSwitchContext\n"
    "    mov r0, #0\n"
    "    msr basepri, r0\n"
    "    ldmia sp!, {r0, r3}\n"
    "    ldr r1, [r3]\n"
    "    ldr r0, [r1]\n"
    "    add r1, r1, #4\n"
    "    ldr r2, =0xe000ed9c\n"
    "    ldmia r1!, {r4-r11}\n"
    "    stmia r2!, {r4-r11}\n"
    "    ldmia r0!, {r3-r11, r14}\n"
    "    msr control, r3\n"
    "    tst r14, #0x10\n"
    "    it eq\n"
    "    vldmiaeq r0!, {s16-s31}\n"
    "    msr psp, r0\n"
    "    bx r14\n"
    "\n"
    "\n"
    "\n" /* Get the location of the current TCB. */
    "\n"
    "\n" /* Is the task using the FPU context? If so, push high VFP registers. */
    "\n"
    "\n" /* stack the FPU context */
    "\n"
    "\n" /* Save the remaining registers. */
    "\n" /* Save the new top of stack into the first member of the TCB. */
    "\n"
    "\n"
    "\n" // errata workaround (-> FreeRTOS CM7 port)
    "\n" //
    "\n" //
    "\n" //
    "\n" // errata workaround (-> FreeRTOS CM7 port)
    "\n" /* call freeRTOS's task scheduler to find next task */
    "\n"
    "\n"
    "\n" /* The first item in the TCB is the task top of stack. */
    "\n" /* Move onto the second item in the TCB... */
    "\n" /* Region Base Address register. */
    "\n" /* Read 4 sets of MPU registers. */
    "\n" /* Write 4 sets of MPU registers. */
    "\n" /* Pop the registers that are not autom. saved on exception entry. */
    "\n"
    "\n" /* Is the task using the FPU context? If so, pop the high VFP registers */
    "\n"
    "\n" /* UNstack the FPU context */
    "\n"
    "\n"
    "\n"
```



Multitasking



Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>

